ЛАБОРАТОРНАЯ РАБОТА № 6

Создание репозитория TaskFlow: .gitignore, первый коммит и ветки

Программное обеспечение: Git; GitHub или Gitflic; VS Code; браузер; доступ в Интернет.

Форма отчётности: ссылка на репозиторий и файл ЛР6_Отчёт_Фамилия.pdf.

Цель: создать репозиторий TaskFlow и освоить базовую последовательность работы Git: подготовка файлов, коммит, отправка и создание веток.

Что предстоит сделать

В этой работе не требуется писать программу или запоминать все команды Git. Вы будете работать с папкой taskflow из предыдущей лабораторной: выполните готовые команды, посмотрите, как меняется состояние репозитория, и объясните результат.

- проверить имя основной ветки и настройки автора коммитов;
- создать удалённый репозиторий TaskFlow;
- заполнить .gitignore и выполнить первый коммит;
- отправить проект и создать две учебные ветки.

Рабочее место и безопасность

Используйте свою папку taskflow. Не удаляйте материалы предыдущих работ. В репозиторий и отчёт нельзя помещать пароли, токены, содержимое файла .env и персональные данные. Если платформа предлагает авторизацию через браузер, следуйте её подсказкам.

Краткая теоретическая справка

Что хранит Git

Git сохраняет историю изменений проекта. Рабочая папка содержит текущие файлы, индекс — выбранные для следующего сохранения изменения, а commit — зафиксированный снимок. Команды git status и git log помогают увидеть состояние и историю.

Удалённый репозиторий и ветки

Remote — связанная копия проекта на сервере; обычно она называется origin. Команда push отправляет локальные коммиты на сервер. Ветка позволяет выполнять отдельную часть работы, не изменяя основную ветку main. В этой работе ветки создаются пустыми: писать в них код пока не нужно.

Понятие	Простое объяснение
commit	сохранённый снимок изменений с сообщением
main	основная ветка проекта
origin	короткое имя удалённого репозитория
push	отправка коммитов на сервер

Ход работы

Шаг 1 Проверьте Git и данные автора

Зачем: Git подписывает каждый коммит именем и электронной почтой автора. Эти данные нужны для истории проекта, а основная ветка должна называться одинаково у всех участников.

Откройте папку taskflow в VS Code, затем во встроенном терминале выполните:

```
git --version
git config --global user.name "Имя Фамилия"
git config --global user.email "email@example.com"
git config --global init.defaultBranch main
git config --global user.name
git config --global user.email
```

Замените пример имени и почты своими данными. Можно использовать адрес no-reply, который предоставляет платформа. В скриншоте для отчёта закройте или размывайте адрес электронной почты.

Ожидаемый результат: команда git --version показывает номер версии, а последние две команды выводят настроенное имя и почту.

Шаг 2 Создайте удалённый репозиторий

Зачем: удалённый репозиторий хранит резервную копию истории и позволяет показать результат преподавателю или работать в команде.

1. Войдите на GitHub или Gitflic и нажмите создание нового репозитория.
2. Укажите имя taskflow. Описание можно оставить коротким: «Учебный проект TaskFlow».
3. Не добавляйте README, .gitignore и лицензию на сайте: эти файлы уже есть локально.
4. Выберите видимость по требованию преподавателя. Для закрытого репозитория заранее выдайте преподавателю доступ.
5. Создайте репозиторий и оставьте страницу с адресом открытой.

Ожидаемый результат: на платформе создан пустой репозиторий taskflow и показан его HTTPS-адрес.

Шаг 3 Инициализируйте локальный репозиторий

Зачем: команда git init добавляет в папку служебную историю Git. Переименование ветки в main устраняет различия между настройками разных версий Git.

```
cd taskflow
git init
git branch -M main
git status
```

Если терминал уже открыт внутри taskflow, строку cd taskflow выполнять не нужно. Команда git status безопасна: она ничего не изменяет, а только показывает состояние.

Ожидаемый результат: Git сообщает, что активна ветка main, а файлы проекта пока не отслеживаются.

Шаг 4 Заполните .gitignore и сделайте первый коммит

Зачем: .gitignore исключает временные, локальные и секретные файлы. Коммит сохраняет только полезное состояние проекта и снабжает его понятным сообщением.

Откройте .gitignore в корне taskflow и вставьте строки:

```
venv/
__pycache__/
*.pyc
.env
.vscode/
.DS_Store
*.sqlite
*.db
```

Запись	Что не попадёт в репозиторий
venv/	установленные локально библиотеки
__pycache__/ и *.pyc	временные файлы Python
.env	локальные настройки и возможные секреты
*.sqlite и *.db	локальные файлы базы данных

Сохраните файл и выполните:

```
git status
git add .
git status
git commit -m "Initial commit: project structure"
git log --oneline -1
```

Сравните два вывода git status: до git add файлы не отслеживаются, после git add они подготовлены к коммиту.

Ожидаемый результат: команда git log --oneline -1 показывает один коммит с сообщением Initial commit: project structure.

Шаг 5 Отправьте проект и создайте ветки

Зачем: push переносит локальную историю на сервер. Отдельные ветки показывают будущие направления работы над API и интерфейсом, не затрагивая main.

Скопируйте HTTPS-адрес своего репозитория и подставьте его вместо примера. Не вставляйте токен или пароль в команду и не сохраняйте их в отчёте.

```
git remote add origin https://ПЛАТФОРМА/ВАШ_ЛОГИН/taskflow.git
git remote -v
git push -u origin main
git switch -c feature/api
git push -u origin feature/api
git switch main
git switch -c feature/frontend
git push -u origin feature/frontend
git switch main
git branch
```

Если origin уже существует, не добавляйте его повторно: выполните git remote -v и проверьте адрес. Если команда git switch не поддерживается, используйте git checkout -b имя_ветки, а для возврата — git checkout main.

1. Откройте репозиторий в браузере и обновите страницу.
2. Убедитесь, что видны файлы проекта и последний коммит.
3. Откройте список веток: должны быть main, feature/api и feature/frontend.
4. Сделайте два скриншота: файлы с коммитом и список веток.

Ожидаемый результат: удалённый репозиторий содержит проект и три ветки; локально активна ветка main.

Если push не выполняется

Сообщение	Что проверить
Authentication failed	вход через браузер или учётные данные платформы
remote origin already exists	адрес командой git remote -v; второй origin не нужен
repository not found	логин, имя taskflow и право доступа

Оформление отчёта

Отчёт должен показывать результат и понимание последовательности Git. Под каждым скриншотом добавьте 1–2 предложения: что проверяли и что подтвердил результат.

Сохраните отчёт под именем ЛР6_Отчёт_Фамилия.pdf. Включите:

1. Титульную часть: Ф. И. О., группа, дата и тема работы.
2. Ссылку на репозиторий и название использованной платформы.
3. Скриншот первого коммита или вывод git log --oneline -1.
4. Скриншот файлов репозитория и пояснение назначения .gitignore.
5. Скриншот списка трёх веток.
6. Ответы на контрольные вопросы и вывод объёмом 3–5 предложений.

В выводе укажите: чем коммит отличается от push, зачем нужен .gitignore и для чего проекту отдельные ветки.

Контрольные вопросы

1. Что сохраняет commit и зачем ему сообщение?
2. Чем git add . отличается от git commit?
3. Зачем файл .env добавляют в .gitignore?
4. Что обозначает имя origin?
5. Почему изменения удобно выполнять в отдельной ветке?

Работа считается выполненной если

1. Git настроен, а основной веткой является main;
2. создан удалённый репозиторий taskflow без секретных данных;
3. .gitignore заполнен и исключает venv, .env и временные файлы;
4. выполнен первый коммит и проект отправлен на сервер;
5. созданы ветки feature/api и feature/frontend, после проверки выполнен возврат в main;
6. в отчёте есть ссылка, необходимые подтверждения, ответы и самостоятельный вывод.

Критерии выставления отметки — согласно сводной таблице 3 (раздел 2.1).

Полезные ссылки

Git: https://git-scm.com/docs; GitHub: https://github.com; Gitflic: https://gitflic.ru.